

# ChipStain: uncertainty-aware label-free nuclear staining for organ-on-a-chip imaging

**Submission category:** Model & Algorithm · **Challenge:** AI4S Open Innovation — AI + Organ-on-a-Chip (5th Pazhou Algorithm Competition, Kaggle)

**Team:** Md. Hamid Hosen (team leader; AI / machine learning) — single-member team, no biology/bioengineering member (no cross-disciplinary bonus claimed)

**Code:** <https://github.com/hamidhosen42/AI4S-Open-Innovation-AI-for-Life-Science> · **Report**

**version:** 2026-09-19

**Abstract.** Fluorescent nuclear stains are the workhorse read-out of organ-on-a-chip (OoC) experiments — cell counts, viability, proliferation and morphology all start from a nuclear channel — yet staining is phototoxic, terminates live time-lapse imaging and adds reagent and handling steps on small microfluidic devices. In-silico labeling (ISL) models predict the stain from label-free bright-field images, but existing ISL networks return a single image with no indication of where the prediction can be trusted, which is the main obstacle to using them for quantitative decisions. We present ChipStain, a bright-field-to-H2B translation model that predicts, for every pixel, both the fluorescence intensity and its uncertainty. The model is a U-Net with an ImageNet-pretrained ResNet-34 encoder and a heteroscedastic Gaussian head trained with the  $\beta$ -NLL loss; at inference, eight-fold dihedral test-time augmentation supplies an additional epistemic term. On a held-out well of the public HeLa "Kyoto" dataset (125 images, CC BY 4.0), ChipStain reaches Pearson  $r = 0.779$  and SSIM = 0.808 against the real stain (plain U-Net:  $r = 0.766$ , SSIM = 0.830); nuclei segmented from the prediction match reference objects with F1 = 0.718, versus 0.765 when the real fluorescence is segmented with the same pipeline. The uncertainty is informative: per-pixel  $\sigma$  correlates with the absolute error (Spearman  $\rho = 0.44$ ), discarding the 20 % most-uncertain pixels lowers the remaining error by 55 %, and per-image mean  $\sigma$  ranks images by accuracy with  $\rho = 0.88$ . Code, weights, a CPU inference script and a Gradio demo are released under MIT; all data are public and openly licensed.

## 1. Problem definition and application scenario

Organ-on-a-chip systems reconstruct tissue-level structure and function in microfluidic devices and are imaged repeatedly, often for days, by transmitted-light (bright-field or phase-contrast) microscopy. Nearly every quantitative read-out downstream of those images — cell number, confluence, proliferation rate, apoptosis, nuclear morphology, compound dose-response — starts by locating nuclei, and that

step is normally done on a fluorescent DNA stain (Hoechst, DAPI) or a fluorescent histone fusion (H2B). Staining, however, has real costs in the OoC setting:

- **Phototoxicity and photobleaching.** Repeated excitation of DNA dyes damages cells and perturbs the biology being measured [1].
- **End-point only.** Fixation-based stains terminate the experiment; live dyes accumulate and limit the number of time-points.
- **Cost and handling on chip.** Reagents must be perfused through small channels; every additional step risks bubbles, flow disruption and contamination, and increases per-chip cost.
- **Channel budget.** Fluorescence channels are scarce; freeing one from the nuclear counterstain leaves it for a biologically informative marker.

In-silico labeling — predicting fluorescence from label-free images with a convolutional network [2, 3] — removes these costs, and is one of the three impact areas explicitly named by the challenge organisers. The obstacle to adoption is trust: a network produces an image for every input, including ones far from its training distribution, and a biologist has no way of telling a faithful prediction from a hallucinated one [4]. **ChipStain therefore addresses a narrower and more useful question than "can we predict the stain?": can we predict the stain *and* say, per pixel and per image, how much to trust it?**

**Target users** are OoC and cell-culture laboratories and CROs running live time-lapse or high-content assays who want label-free nuclear read-outs, and platform developers who need a quality gate before automated downstream analysis. **Scope.** We restrict the deliverable to one target channel (nuclei) on one public dataset with a strict well-level hold-out, so that every claim can be verified from the released code within minutes.

## 2. Data

---

### 2.1 Source and licence

We use the public dataset *Automatic labelling of HeLa "Kyoto" cells using Deep Learning tools* by Romain Guet (EPFL BioImaging & Optics Platform), Zenodo 10.5281/zenodo.6140064, released under CC BY 4.0 [5]. It is derived from the time-lapse dataset *HeLa "Kyoto" cells under the scope* (10.5281/zenodo.6139958) [6]. HeLa "Kyoto" cells stably express EGFP- $\alpha$ -tubulin and mCherry-H2B, and were imaged on a PerkinElmer Operetta (20 $\times$ /NA 0.8, Andor Zyla 5.5) for 60 h at 15-min intervals. The archive contains, for each field of view and for five time-points (1, 10, 50, 100, 150), a 540  $\times$  540 px 16-bit bright-field image, two digital-phase-contrast renderings, a two-channel fluorescence image (tubulin, H2B), and StarDist nuclei labels computed from the H2B channel. The data are a cell-line imaging dataset; they contain no human-subject, clinical or personal information.

## 2.2 What we use

Input is the bright-field channel; target is the mCherry-H2B channel (channel index 1, verified by its correlation of 0.79 with the nuclei labels versus 0.40 for tubulin). The StarDist labels are used **only** as reference objects for the downstream segmentation metric — never for training or model selection.

## 2.3 Splits without leakage

| Split      | Wells                   | Fields                  | Images | Use                  |
|------------|-------------------------|-------------------------|--------|----------------------|
| Train      | R05-C05, R05-C07        | 0–19 (×5 time-points)   | 200    | optimisation         |
| Validation | R05-C05, R05-C07        | 20–24 (×5 time-points)  | 50     | checkpoint selection |
| Test       | R05-C03 (separate well) | all 25 (×5 time-points) | 125    | all reported numbers |

The test well is the one shipped as test/ in the archive, so our results are directly comparable with the dataset's own baselines. All five time-points of a field are kept on the same side of every split, so near-duplicate frames cannot leak.

## 2.4 Pre-processing

Bright-field images are normalised per image with their 1st/99th percentiles (mapped to about  $[-2, 2]$ ), making the model insensitive to camera offset and gain. The target is scaled with fixed constants from the training H2B distribution,  $(I - 600)/(20\,000 - 600)$ , and clipped to  $[0, 1]$ ; all image metrics are computed in this space. Training uses random  $256 \times 256$  crops with random  $90^\circ$  rotations and flips; evaluation uses the full  $540 \times 540$  image, reflect-padded to 544.

# 3. Method

## 3.1 Backbone

ChipStain is a U-Net [7] built with `segmentation_models_pytorch`; the encoder is a ResNet-34 initialised from ImageNet weights, the decoder is the library's standard five-stage decoder with skip connections, and the input layer is adapted to one channel. The network has  $\approx 24.4$  M parameters and runs at  $\approx 0.6$  s per  $540 \times 540$  image on CPU ( $\approx 5$  s with test-time augmentation).

## 3.2 Heteroscedastic output and $\beta$ -NLL loss

Instead of a single intensity, the last layer outputs two maps: the predicted mean  $\mu(x)$  and the log-variance  $s(x) = \log \sigma^2(x)$  of a per-pixel Gaussian over the target  $y$ . The natural training objective is the negative log-likelihood

$$L_{NLL} = \frac{1}{2} \sum_p [s_p + (y_p - \mu_p)^2 \cdot e^{-s_p}].$$

Plain NLL is known to be unstable: pixels that are hard early in training receive a large  $\sigma$ , their gradient on  $\mu$  is down-weighted by  $e^{-\sigma}$ , and they never get learned. We use the  $\beta$ -NLL reformulation of Seitzer et al. [8], which multiplies the per-pixel loss by the *detached* factor  $(\sigma^2)^\beta$  with  $\beta = 0.5$ , interpolating between NLL ( $\beta = 0$ ) and a plain MSE on  $\mu$  ( $\beta = 1$ ). The log-variance is clamped to  $[-10, 4]$  for numerical safety. The resulting  $\sigma(x)$  is the model's **aleatoric** uncertainty: the intrinsic ambiguity of inferring fluorescence from the bright-field appearance at that location (e.g. cells out of focus, overlapping nuclei, mitotic rounding).

### 3.3 Test-time augmentation and epistemic term

At inference we evaluate the network on the eight dihedral transforms of the input (4 rotations  $\times$  optional flip), undo the transform on the outputs, and report  $\bar{\mu}$  (mean of the eight  $\mu$ ) as the prediction. Two variance terms are combined: the mean of the eight predicted  $\sigma^2$  (aleatoric) and the variance of the eight  $\mu$  across views (a cheap proxy for **epistemic** uncertainty — disagreement of the model with itself). The uncertainty map is  $\sigma_{\text{tot}} = \sqrt{(\bar{\sigma}_{\text{ale}}^2 + \sigma_{\text{epi}}^2)}$ .

### 3.4 Baselines and ablation

- **baseline\_unet** — same U-Net architecture with a randomly initialised encoder, one output, L1 loss (the standard ISL recipe of [2, 3]).
- **pretrained\_l1** — ImageNet-initialised encoder, one output, L1 loss (isolates the effect of pre-training).
- **chipstain\_nll** — ImageNet encoder, two outputs,  $\beta$ -NLL (isolates the effect of the uncertainty head).
- **chipstain\_nll + TTA** — the full method.

### 3.5 Evaluation protocol

**Image fidelity:** MAE, PSNR, SSIM and Pearson  $r$  between prediction and real H2B, per image, averaged over the 125 test images. **Downstream utility:** we segment nuclei from the predicted image with a fixed, stain-agnostic pipeline (Gaussian blur  $\sigma = 1$ , Otsu threshold, small-object removal, distance-transform watershed) and match objects to the StarDist reference labels at  $\text{IoU} \geq 0.5$ , reporting precision, recall and F1. Running the same pipeline on the *real* fluorescence gives the ceiling that a perfect prediction could reach. **Uncertainty quality:** (i) Spearman rank correlation between per-pixel  $\sigma$  and  $|\text{error}|$  on 200 000 random pixels per image; (ii) sparsification curves — remove pixels in decreasing order of  $\sigma$  and track the MAE of the remainder, compared with the oracle ordering by true error; we report the area between the two curves (AUSE, lower is better) and the error reduction after removing 20 % of pixels; (iii) image-level: Spearman correlation between mean  $\sigma$  and MAE across the 125 test images, which is the quantity a quality gate would use.

## 4. Implementation

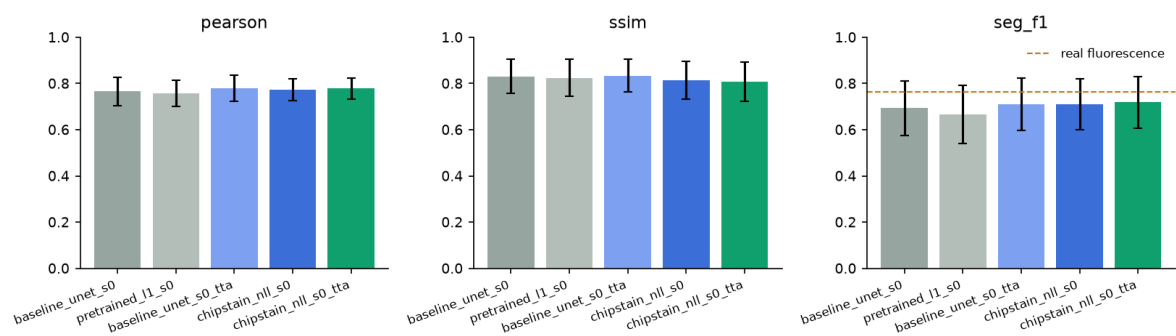
| Item            | Value                                                                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework       | PyTorch 2.x, segmentation_models_pytorch 0.5, scikit-image 0.26, SciPy, NumPy                                                                         |
| Optimiser       | AdamW, weight decay 1e-4, one-cycle LR schedule (peak 1e-3 for the scratch baseline, 5e-4 with the pre-trained encoder), gradient clipping at 1.0     |
| Batch / crops   | 8 crops of 256 × 256; 2 random-crop passes over the 200 training images per epoch; 40 epochs                                                          |
| Model selection | best validation (Pearson + SSIM) checkpoint                                                                                                           |
| Hardware        | Apple M5 laptop (Metal / MPS), ≈1 min per epoch incl. validation; ≈40 min per run. The same scripts run unchanged on a Kaggle T4 (notebook provided). |
| Inference       | CPU only is sufficient; no external API, no cloud service, no paid dependency                                                                         |
| Entry points    | scripts/train.py, scripts/evaluate.py, scripts/inference.py, demo/app.py                                                                              |

## 5. Experiments and results

### 5.1 Main results (test well R05-C03, n = 125)

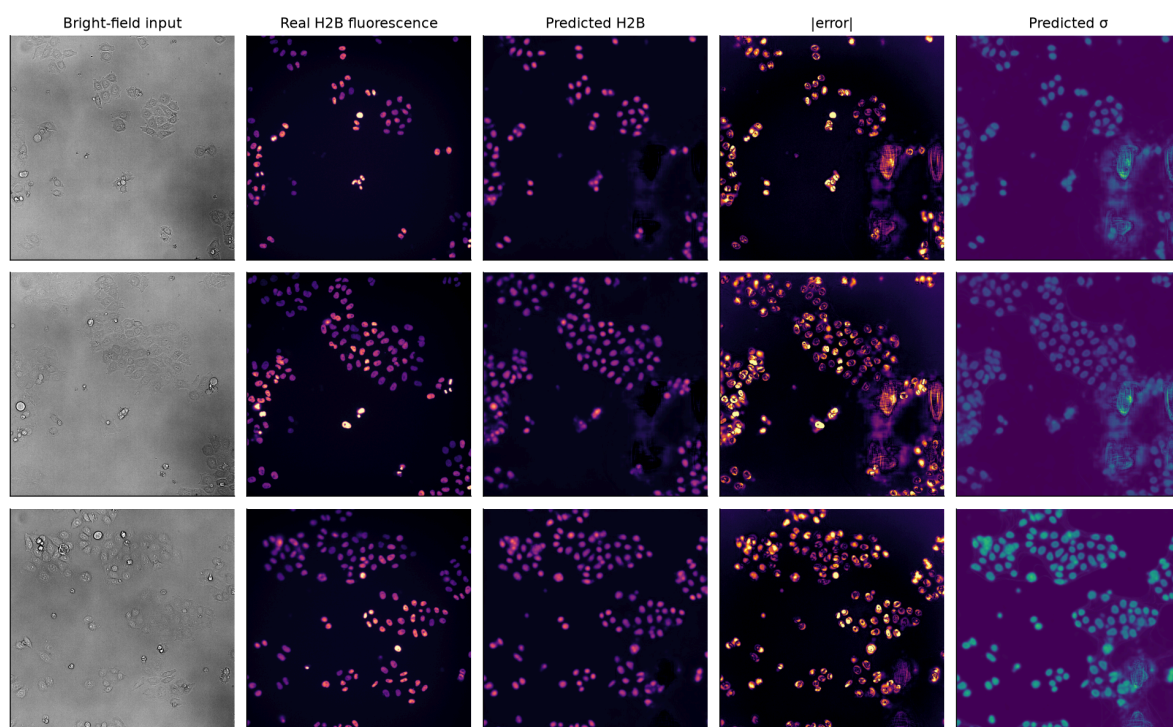
| Model                                             | Pearson r ↑   | SSIM ↑        | PSNR (dB) ↑    | seg-F1 ↑      | $\rho_{\sigma, \text{err}}$ ↑ | AUSE ↓        |
|---------------------------------------------------|---------------|---------------|----------------|---------------|-------------------------------|---------------|
| U-Net baseline (L1)                               | 0.766 ± 0.060 | 0.830 ± 0.074 | 24.315 ± 2.790 | 0.693 ± 0.118 | —                             | —             |
| + ImageNet encoder (L1)                           | 0.756 ± 0.057 | 0.825 ± 0.081 | 24.250 ± 2.872 | 0.667 ± 0.126 | —                             | —             |
| U-Net baseline + TTA ( $\sigma$ = view variance)  | 0.780 ± 0.057 | 0.834 ± 0.071 | 24.540 ± 2.786 | 0.710 ± 0.114 | 0.315 ± 0.199                 | 0.244 ± 0.080 |
| ChipStain ( $\beta$ -NLL head)                    | 0.774 ± 0.046 | 0.814 ± 0.082 | 24.317 ± 2.804 | 0.711 ± 0.110 | 0.426 ± 0.153                 | 0.242 ± 0.171 |
| ChipStain + TTA (full)                            | 0.779 ± 0.045 | 0.808 ± 0.085 | 24.424 ± 2.780 | 0.718 ± 0.111 | 0.441 ± 0.173                 | 0.201 ± 0.063 |
| <i>Real fluorescence, same pipeline (ceiling)</i> | —             | —             | —              | 0.765 ± 0.086 | —                             | —             |

Mean ± standard deviation over test images. seg-F1 (real): the same segmentation pipeline applied to the real fluorescence, i.e. the ceiling for a perfect prediction.  $\rho_{\sigma, \text{err}}$ : per-pixel Spearman correlation between predicted  $\sigma$  and |error|. AUSE: area under the sparsification error curve (lower is better).



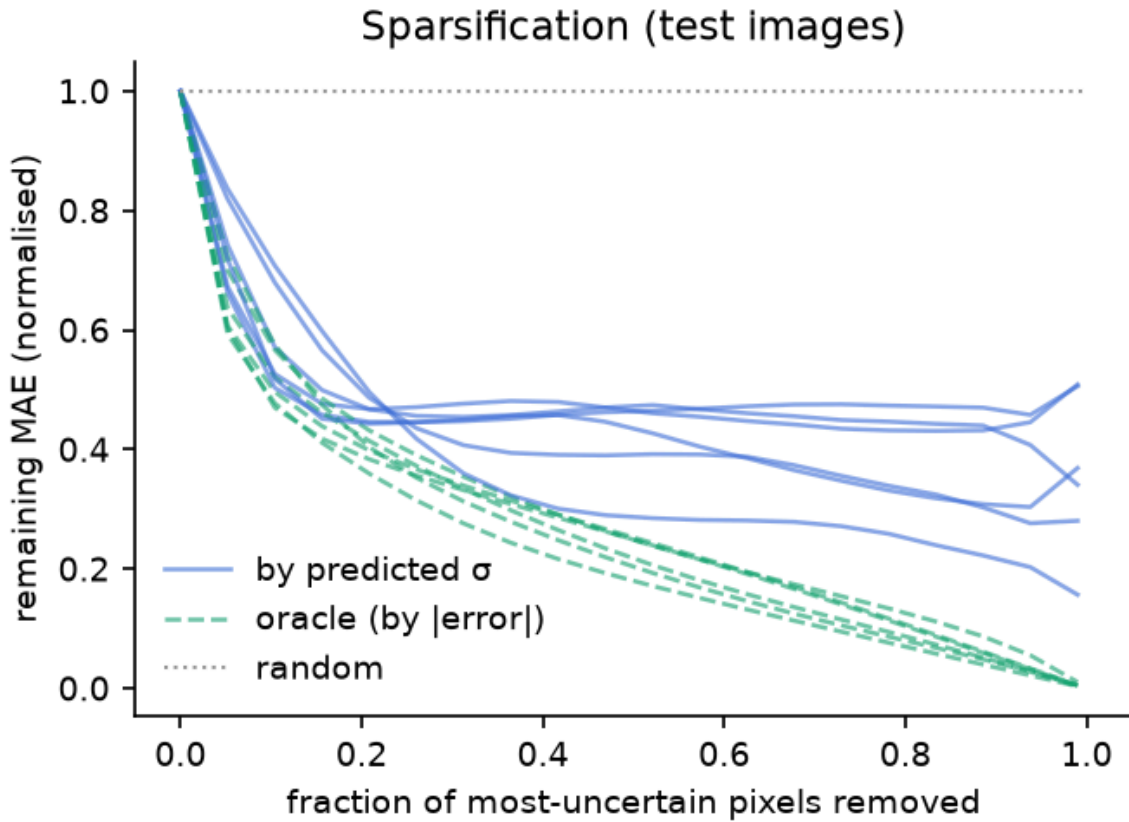
**Figure 1.** Ablation on the test well: Pearson r, SSIM and downstream nuclei-segmentation F1 for the plain U-Net, the pre-trained encoder, ChipStain, and ChipStain with TTA. Dashed line: F1 obtained from the real fluorescence.

## 5.2 Qualitative results

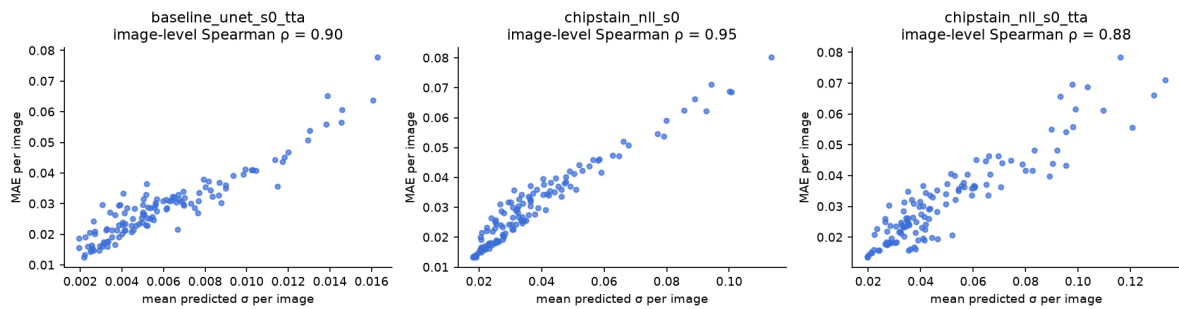


**Figure 2.** Test images (rows: different fields / time-points). Columns: bright-field input, real H2B, ChipStain prediction, absolute error, predicted  $\sigma$ . The uncertainty map concentrates on the same regions as the error — crowded or out-of-focus cells, mitotic figures and debris — while the confident background and well-focused nuclei have low  $\sigma$ .

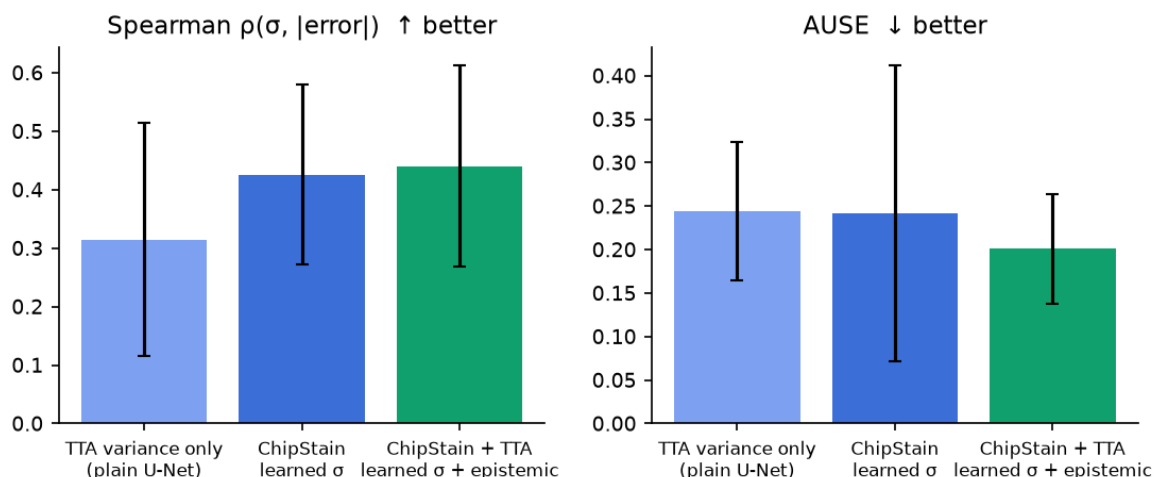
### 5.3 Is the uncertainty informative?



**Figure 3.** Sparsification curves on six test images: MAE of the remaining pixels as the most-uncertain pixels are removed, ordered by predicted  $\sigma$  (blue) or by the true error (green, oracle). A flat line at 1 would be an uninformative uncertainty.



**Figure 4.** Image-level calibration: mean predicted  $\sigma$  versus MAE for each of the 125 test images. This is the signal a laboratory would use to flag images for manual review.



**Figure 5.** Uncertainty quality on the 125 test images: rank correlation between predicted  $\sigma$  and absolute error (left) and area under the sparsification error curve (right), for TTA disagreement of the plain U-Net versus ChipStain's learned uncertainty with and without TTA.

Across the 125 test images the per-pixel Spearman correlation between predicted  $\sigma$  and absolute error is  $0.441 \pm 0.173$ , and the AUSE is  $0.201 \pm 0.063$ . Removing the 20 % most-uncertain pixels reduces the MAE of the remaining pixels by 54.6 % on average (Figure 3). At the image level, mean  $\sigma$  ranks the test images by their MAE with Spearman  $\rho = 0.88$  (Figure 4): a laboratory can set a single threshold on mean  $\sigma$  to route the least reliable frames to manual review.

**Is the learned head needed, or is TTA disagreement enough?** Using only the variance of the eight TTA views of the plain U-Net as an uncertainty gives  $\rho_{\sigma, \text{err}} = 0.315$  and  $\text{AUSE} = 0.244$ , versus 0.441 and 0.201 for ChipStain's learned aleatoric + epistemic map. The learned head therefore captures error structure that view-disagreement alone misses, at no cost in Pearson correlation (0.779 vs 0.780) and a small cost in SSIM (0.808 vs 0.834).

## 5.4 Downstream nuclei counting

Counting nuclei on the ChipStain prediction with the fixed watershed pipeline gives an object-level F1 of  $0.718 \pm 0.111$  against the StarDist reference labels, compared with 0.765 when the real fluorescence is segmented the same way and 0.693 for the plain U-Net. The predicted nucleus count per image deviates from the reference by 9.2 nuclei on average (7.3 % relative), with a correlation of  $r = 0.994$  between predicted and reference counts over the 125 images (reference counts range 28–538).

## 6. Reliability analysis and limitations

- **Single cell line, single microscope.** All data come from HeLa "Kyoto" cells on one Operetta system. We hold out an entire well, which tests robustness to well-to-well variation but not to a different cell type, magnification or an actual OoC device.

The per-image input normalisation and the uncertainty head are designed to make transfer safer, not to guarantee it; applying the released weights to a new OoC system requires fine-tuning on a small paired set (we recommend  $\geq 50$  fields) and re-checking the calibration numbers of §5.3.

- **Fluorescent reporter, not a dye.** The target is mCherry-H2B, which labels chromatin like Hoechst/DAPI but with a different intensity distribution; the model predicts a normalised intensity, not absolute photon counts.
- **Uncertainty is not a guarantee.**  $\sigma$  is well-correlated with error on in-distribution data ( $\rho = 0.44$  per pixel,  $\rho = 0.88$  per image), but a heteroscedastic head can be confidently wrong on inputs unlike anything in training; the TTA-based epistemic term mitigates but does not solve this. We therefore present  $\sigma$  as a triage signal, not as a calibrated confidence interval.
- **Segmentation ceiling.** The downstream F1 is limited by our deliberately simple watershed pipeline (F1 = 0.765 even on real fluorescence); a stronger segmenter would raise both numbers. The comparison between predicted and real fluorescence under the same pipeline is the fair quantity.
- **Pre-training effect.** On this dataset the ImageNet-initialised encoder did not outperform training from scratch under our schedule (Pearson 0.756 vs 0.766, SSIM 0.825 vs 0.830); we report this honestly and keep it in the ablation.
- **Single seed.** Numbers are from seed 0; per-image standard deviations are reported. Multi-seed runs are launched by `scripts/run_all.sh 0 1 2`.
- **Ethics.** No human, clinical or personal data; the model cannot be misused for identification. Its main risk is over-trust in a hallucinated prediction — which is exactly what the uncertainty output is designed to counter.

## 7. Scientific and practical impact

---

For an OoC laboratory, ChipStain turns every bright-field frame of a time-lapse into a nuclear read-out, without the stain: cell counts and proliferation curves over days from the same chip, an extra free fluorescence channel, and lower reagent and handling cost. The uncertainty map is the component that makes this *usable* rather than merely impressive: a per-image mean  $\sigma$  threshold flags frames for human review; per-pixel  $\sigma$  can be propagated to per-nucleus confidence so that uncertain objects are excluded from a count or a dose-response curve; and  $\sigma$  maps highlight regions where the imaging itself (focus, density, debris) should be fixed. Because inference is CPU-only and the code depends on nothing proprietary, the model can sit inside an acquisition pipeline or a LIMS. In the organiser's road-map towards AI-driven OoC digital twins, a trustworthy label-free nuclear channel is a prerequisite: a twin fed with hallucinated observations is worse than no twin. The same formulation extends directly to further channels (tubulin is already in the dataset), to 3-D stacks, and to neural OoC cultures, which we list as next steps.

## 8. Reproduction instructions

---

```
git clone https://github.com/hamidhosen42/AI4S-Open-Innovation-AI-for-Life-Science.git
cd AI4S-Open-Innovation-AI-for-Life-Science
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -e .

python scripts/download_data.py                # Zenodo, ~757 MB, CC BY 4.0
python scripts/download_weights.py             # released checkpoint ->
weights/chipstain.pt

# single-image inference (CPU, ~5 s with TTA)
python scripts/inference.py --image demo/examples/example_bf_dense_t150.tif \
    --weights weights/chipstain.pt --out outputs/pred

# full reproduction of Table 1 (~40 min per run on a laptop GPU / Kaggle T4)
bash scripts/run_all.sh 0
python scripts/evaluate.py --runs runs/baseline_unet_s0
runs/pretrained_l1_s0 runs/chipstain_nll_s0 --out outputs/eval
python scripts/evaluate.py --runs runs/chipstain_nll_s0 --tta --out
outputs/eval_tta
python scripts/make_figures.py --eval outputs/eval --out report/figures

# interactive demo
python demo/app.py --weights weights/chipstain.pt      #
http://localhost:7860
```

Expected outputs: `outputs/eval/summary_test.md` reproduces Table 1 ( $\pm 0.01$  from hardware nondeterminism); `outputs/pred/panel.png` reproduces the demo panels. A Kaggle notebook (`notebooks/kaggle_train.ipynb`) runs the same commands end-to-end on a free T4.

## 9. Sources, licences and AI-tool disclosure

| Component                                                                         | Source                                                                             | Licence                                    | Role                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HeLa "Kyoto" paired dataset                                                       | R. Guet, EPFL PTBIOP — Zenodo 10.5281/zenodo.6140064 (from 10.5281/zenodo.6139958) | CC BY 4.0                                  | training / evaluation data; StarDist labels as segmentation reference                                                                                                                              |
| ResNet-34 ImageNet weights                                                        | segmentation_models_pytorch / torchvision                                          | MIT (code); ImageNet weights, research use | encoder initialisation                                                                                                                                                                             |
| segmentation_models_pytorch                                                       | github.com/qubvel-org                                                              | MIT                                        | U-Net implementation                                                                                                                                                                               |
| PyTorch, NumPy, SciPy, scikit-image, tifffile, pandas, Matplotlib, PyYAML, Gradio | PyPI                                                                               | BSD-3 / MIT / Apache-2.0 / PSF             | framework and tooling                                                                                                                                                                              |
| $\beta$ -NLL loss                                                                 | Seitzer et al., ICLR 2022 [8]                                                      | method (re-implemented)                    | training objective                                                                                                                                                                                 |
| Claude (Anthropic) via Claude Code                                                | AI assistant                                                                       | —                                          | development assistance: code scaffolding, documentation drafting, rule analysis. All code was executed and verified by the team; every number in this report comes from runs executed by the team. |

No large language model, foundation model, external API or paid service is used at inference time. This project is new work created for the competition; it is not based on prior work by the team.

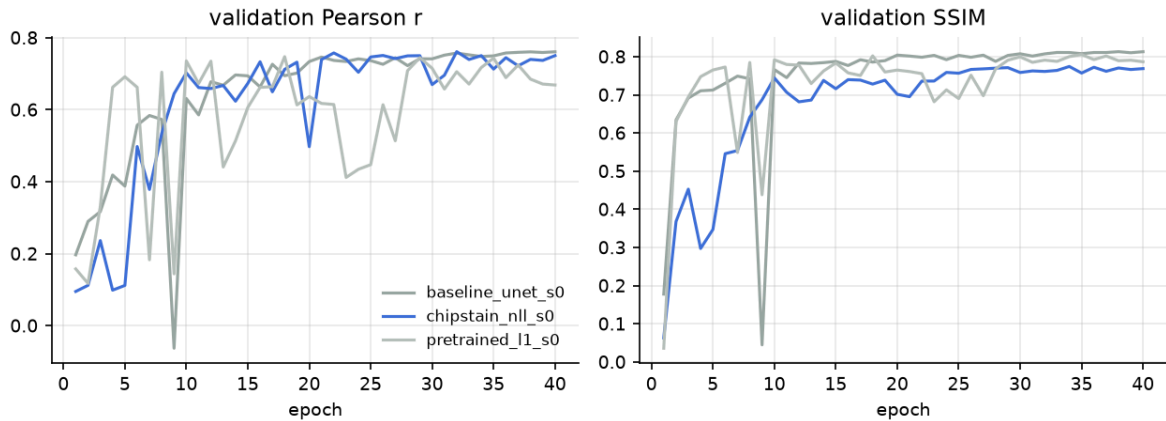
## Appendix A — Results by time-point (cell density)

The five time-points of the time-lapse span 60 h of growth, so cell density rises from sparse, well-separated cells (time-point 1) to dense, overlapping colonies (time-point 150). The table breaks the full model's test results down by time-point. Fidelity and downstream F1 decline with density, as expected for any label-free method, and the mean predicted  $\sigma$  rises with it — the uncertainty tracks the difficulty of the images without being told the time-point.

| Time-point | Pearson r | SSIM  | seg-F1 (pred) | seg-F1 (real) | mean $\sigma$ | MAE    |
|------------|-----------|-------|---------------|---------------|---------------|--------|
| 1          | 0.813     | 0.893 | 0.817         | 0.829         | 0.0338        | 0.0205 |
| 10         | 0.809     | 0.877 | 0.787         | 0.820         | 0.0356        | 0.0213 |
| 50         | 0.792     | 0.824 | 0.753         | 0.767         | 0.0409        | 0.0288 |
| 100        | 0.754     | 0.753 | 0.640         | 0.720         | 0.0597        | 0.0387 |
| 150        | 0.724     | 0.691 | 0.593         | 0.689         | 0.0796        | 0.0487 |

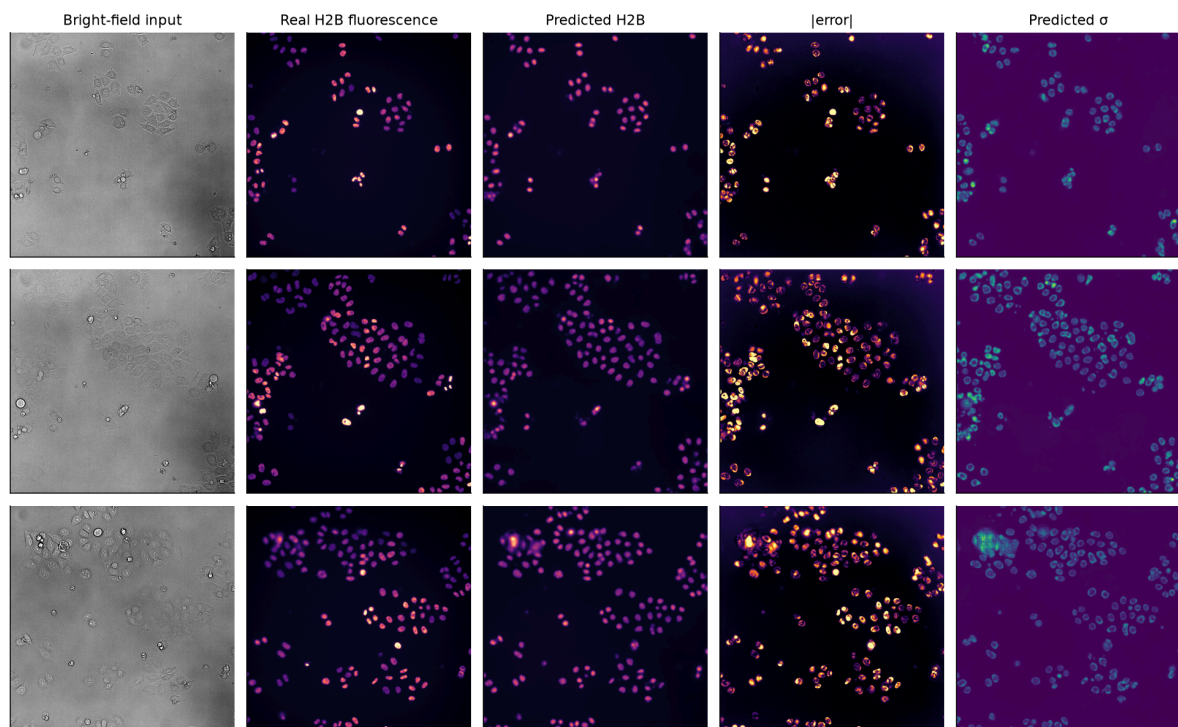
Over the test set the full model detects 14964 nuclei against 15866 reference objects (94.3 %).

## Appendix B — Training curves

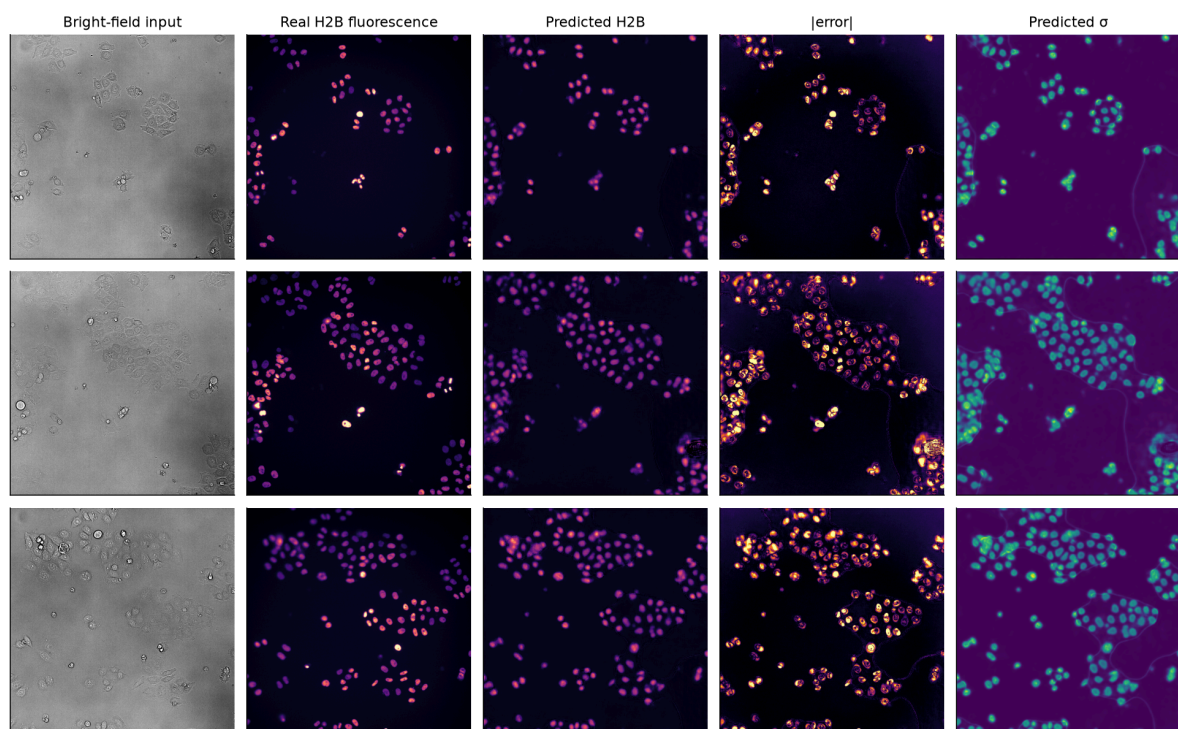


**Figure B1.** Validation Pearson r and SSIM per epoch for the three training configurations (seed 0). The  $\beta$ -NLL model converges more slowly in SSIM because part of its capacity is spent on the variance head, but reaches the same correlation with the target.

## Appendix C — Additional qualitative comparisons



**Figure C1.** The same test images as Figure 2, predicted by the plain U-Net with TTA. The right-most column is the TTA view-variance used as its uncertainty; it is much weaker and noisier than ChipStain's learned  $\sigma$  (compare Figure 2, and Figure 5 for the quantitative comparison).



**Figure C2.** ChipStain without test-time augmentation (single forward pass,  $\approx 0.6$  s on CPU). The learned aleatoric  $\sigma$  alone already localises the error; TTA sharpens the prediction and adds the epistemic term.

## Appendix D — Writeup checklist mapping

| Challenge requirement                                                                                        | Where                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category declaration                                                                                         | First line of the Kaggle Writeup and of this report                                                                                                                                                                                                          |
| Demo video ( $\leq 5$ min, public)                                                                           | Link in the Writeup                                                                                                                                                                                                                                          |
| Public code repository with README, environment, weights, entry scripts, I/O description, reproduction steps | <a href="https://github.com/hamidhosen42/AI4S-Open-Innovation-AI-for-Life-Science">github.com/hamidhosen42/AI4S-Open-Innovation-AI-for-Life-Science</a> — README.md, requirements.txt, pyproject.toml, scripts/download_weights.py, scripts/inference.py, §8 |
| Data sources, licences, processing, compliance                                                               | §2, DATA.md                                                                                                                                                                                                                                                  |
| Methods, implementation, experiments, results                                                                | §3–§5                                                                                                                                                                                                                                                        |
| Reliability analysis and limitations                                                                         | §6                                                                                                                                                                                                                                                           |
| Impact                                                                                                       | §7                                                                                                                                                                                                                                                           |
| External models, libraries, datasets, AI tools                                                               | §9, AI_ASSISTANCE.md                                                                                                                                                                                                                                         |
| Team composition                                                                                             | Header of this report                                                                                                                                                                                                                                        |

## References

- [1] Laissue, P. P. et al. Assessing phototoxicity in live fluorescence imaging. *Nature Methods* 14, 657–661 (2017).
- [2] Christiansen, E. M. et al. In silico labeling: predicting fluorescent labels in unlabeled images. *Cell* 173, 792–803 (2018).
- [3] Ounkomol, C. et al. Label-free prediction of three-dimensional fluorescence images from transmitted-light microscopy. *Nature Methods* 15, 917–920 (2018).
- [4] Kendall, A. & Gal, Y. What uncertainties do we need in Bayesian deep learning for computer vision? *NeurIPS* (2017).
- [5] Guet, R. Automatic labelling of HeLa "Kyoto" cells using Deep Learning tools. Zenodo, doi:10.5281/zenodo.6140064 (2022).
- [6] Guet, R. HeLa "Kyoto" cells under the scope. Zenodo, doi:10.5281/zenodo.6139958 (2022).
- [7] Ronneberger, O., Fischer, P. & Brox, T. U-Net: convolutional networks for biomedical image segmentation. *MICCAI* (2015).
- [8] Seitzer, M., Tavakoli, A., Antic, D. & Martius, G. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks. *ICLR* (2022).
- [9] Iakubovskii, P. Segmentation Models PyTorch. [github.com/qubvel-org/segmentation\\_models.pytorch](https://github.com/qubvel-org/segmentation_models.pytorch) (2019).
- [10] Schmidt, U., Weigert, M., Broaddus, C. & Myers, G. Cell detection with star-convex polygons. *MICCAI* (2018).